

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Вариант 3

Студент гр. 4343

Лапшин А. Е.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

1 задание.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 1000000$).

Вторая – число $n (1 \leq n \leq 3000)$, каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел – i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

2 задание.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Вход:

Текст (T, $1 \leq |T| \leq 1000000$)

Шаблон (P, $1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Индивидуальное задания.

Вариант 3. Вычислить длину самой длинной цепочки из суффиксных ссылок и самой длинной цепочки из конечных ссылок в автомате.

Выполнение работы.

Алгоритм Ахо-Корасик

Бор - это древовидная структура данных для хранения множества строк. Каждая вершина соответствует некоторому префиксу одной или нескольких строк из набора. Рёбра помечены символами алфавита.

Суффиксная ссылка для вершины v , представляющей строку S , суффиксная ссылка $suffix_link[v]$ указывает на вершину, представляющую **наибольший собственный суффикс** строки S , присутствующий в боре. При поиске, если из текущей вершины нет перехода по символу текста, мы не возвращаемся в корень, а переходим по суффиксной ссылке. Это позволяет не терять уже найденные частичные совпадения.

Конечная ссылка ***final_link[v]*** указывает на **ближайшую терминальную вершину** по цепочке суффиксных ссылок. Если таких нет, то на корень. При поиске, попав в вершину, нужно собрать все образцы, заканчивающиеся в этой позиции. Без конечных ссылок пришлось бы идти по всей цепочке суффиксных ссылок и проверять каждую вершину. Конечные ссылки позволяют переходить сразу к следующему терминалу, пропуская нетерминальные вершины.

Автомат - это бор, расширенный суффиксными и конечными ссылками, а также функцией перехода ***go(v, c)***.

Функция перехода $go(v, c)$ определяет, в какую вершину перейти из v по символу c :

1. Если есть прямой переход по бору — возвращает его.
2. Если v — корень — остаётся в корне.
3. Иначе рекурсивно переходит по суффиксной ссылке:
go(suffix_link[v], c).

Поиск. Автомат обрабатывает текст посимвольно. Для каждого символа выполняется переход $v = go(v, c)$. Затем по цепочке конечных ссылок собираются все образцы, заканчивающиеся в текущей позиции.

задание 1 – множественный поиск образцов

Хранение автомата. Автомат представлен массивом вершин. Каждая вершина хранит:

1. массив переходов по пяти символам алфавита $\{A, C, G, T, N\}$
2. суффиксную ссылку (индекс вершины с наибольшим суффиксом)
3. конечную ссылку (индекс ближайшей терминальной вершины по цепочке суффиксных ссылок)
4. индекс родительской вершины и символ, по которому в неё пришли
5. идентификатор образца

Построение бора. Для каждого образца программа проходит от корня по символам. Если нужное ребро отсутствует, создаётся новая вершина, которая связывается с родителем. В конце пути в последнюю вершину записывается номер образца. Таким образом, все образцы добавляются в общее префиксное дерево.

Построение автомата (суффиксных и конечных ссылок). Используется обход бора в ширину с помощью очереди. Для корня все отсутствующие переходы зацикливаются на него же. Дети корня получают суффиксную и конечную ссылки, равные корню, и помещаются в очередь. Далее для каждой вершины из очереди обрабатываются все её дети. Для каждого ребёнка *child* по символу *c*:

- Суффиксная ссылка вычисляется подъёмом по суффиксным ссылкам родителя, пока не встретится вершина с переходом по *c*. Найденный переход становится суффиксной ссылкой ребёнка.
- Конечная ссылка определяется по уже вычисленной суффиксной ссылке ребёнка: если та указывает на терминал, то конечная ссылка равна ей; если указывает на корень, то конечная ссылка равна корню; иначе конечная ссылка наследуется от конечной ссылки суффиксной ссылки ребёнка.

Поиск образцов в тексте. Начиная с корня, для каждого символа текста выполняется переход $v = go(v, c)$. Затем по цепочке конечных ссылок, начиная с v , собираются все терминальные вершины. Для каждого найденного терминала вычисляется начальная позиция образца (текущий индекс конца образца минус его длина плюс один). Результат (позиция, номер образца) сохраняется.

Вывод результатов. Найденные вхождения сортируются по позиции, а при равных позициях по номеру образца. Нумерация позиций и образцов в выводе начинается с единицы.

Сложность.

Построение бора. Каждый символ каждого образца обрабатывается один раз — или создаётся новая вершина, или происходит переход по существующему ребру. Создание вершины и добавление ребра выполняется за $O(1)$. Суммарно $O(m)$, где m - суммарная длина всех образцов.

Построение автомата (суффиксных и конечных ссылок). BFS посещает каждую вершину один раз. Для каждой вершины перебираются все 5 символов алфавита. Цикл while при вычислении суффиксной ссылки в сумме по всем вершинам делает не более $O(m)$ итераций (каждая итерация уменьшает глубину, а суммарное увеличение глубины ограничено количеством рёбер). Итог $O(m \cdot |\Sigma|) = O(5m) = O(m)$.

Поиск в тексте. Для каждого из L символов текста вызывается функция go . В худшем случае (например, текст из одного повторяющегося символа, не встречающегося в образцах) каждый вызов go поднимается до корня по цепочке длиной $O(h)$, где h - максимальная глубина бора. Глубина бора не превышает максимальной длины шаблона. Значит, один вызов go - $O(h)$. Для всех L символов - $O(L \cdot h)$.

Сортировка результатов. В худшем случае каждый образец может начинаться в каждой позиции текста. Количество вхождений $k \leq L \cdot n$. Сортировка $O(k \log k) = O(L \cdot n \cdot \log(L \cdot n))$.

Итог: $O(L \cdot n \cdot \log(L \cdot n) + 6m)$

Пространственная сложность: $O(m + L + k)$, где $k \leq L \cdot n$ - количество найденных вхождений. Основной вклад даёт бор $O(m)$.

Задание 2. – поиск с джокером

Отличия в структуре вершины. Поскольку в задаче с джокером один и тот же фрагмент (например, одиночная буква) может встречаться в шаблоне несколько раз на разных позициях, одна вершина бора может соответствовать нескольким фрагментам. Поэтому вместо одного идентификатора образца вершина хранит список идентификаторов фрагментов.

Разбиение шаблона. Шаблон сканируется слева направо. Все максимальные подстроки, не содержащие символа-джокера, выделяются в отдельные фрагменты. Для каждого фрагмента запоминается его начальная позиция в исходном шаблоне.

Построение бора и автомата выполняется точно так же, как в базовой программе, но с учётом того, что в терминальную вершину может быть добавлено несколько идентификаторов фрагментов.

Поиск с джокерами. Заводится массив счётчиков *matches* размером с длину текста, изначально заполненный нулями. Для каждой позиции текста выполняется переход автомата. Затем по цепочке конечных ссылок перебираются все найденные терминальные вершины и для каждого фрагмента, хранящегося в них, вычисляется предполагаемая начальная позиция всего шаблона. Формула: $\text{начало} = \text{текущая_позиция} - \text{длина_фрагмента} - \text{позиция_фрагмента_в_шаблоне} + 1$. Если начало лежит в допустимых границах (неотрицательно и шаблон полностью помещается в текст), соответствующий счётчик увеличивается. Когда счётчик для некоторой позиции становится равным общему количеству фрагментов, это означает, что все фрагменты совпали на своих местах, и номер позиции выводится.

Вывод. Номера начальных позиций найденных вхождений шаблона выводятся по одному на строку в порядке возрастания.

- Текст T , длина L .
- Шаблон P , длина $|P|$.
- Алфавит Σ , размер $|\Sigma|$ (константа, 5 символов).

Разбиение шаблона. Один проход по шаблону - $O(|P|)$.

Построение бора и автомата. Суммарная длина фрагментов не превышает $|P|$. Количество вершин $O(|P|)$. Построение бора $O(|P|)$. Построение автомата $O(|P|)$.

Поиск в тексте. Для каждого символа вызывается go . Глубина бора не превышает L , поэтому один вызов go $O(L)$ (в худшем случае подъём по всем суффиксным ссылкам). Итого $O(L \cdot |P|)$.

Итоговая временная сложность: $O(|P| + L \cdot |P|) = O(L \cdot |P|)$.

Пространственная сложность: бор - $O(|P|)$ вершин, массив счётчиков - $O(L)$, текст - $O(L)$. Итого $O(L + |P|)$.

Индивидуальное задание – максимальные длины цепочек ссылок

Максимальная цепочка суффиксных ссылок. Для каждой вершины автомата (кроме корня) подсчитывается длина пути по суффиксным ссылкам до корня: пока текущая вершина не корень, увеличивается счётчик и совершается переход по суффиксной ссылке. Из всех полученных длин выбирается максимальная.

Максимальная цепочка конечных ссылок. Аналогично, для каждой вершины измеряется количество переходов по конечным ссылкам до достижения корня. Наибольшее значение сохраняется.

Так как это задание дополняет первое задание, то сложность будет аналогичная, поскольку проход по все ссылкам дает незначительный прирост сложности.

Тестирование

Таблица 1 - Результаты тестирования для задания 1

Тест	Входные данные	Выходные данные
1	AAAA 2 AA CC	1 1 2 1 3 1
2	NTAG 3 TAGT TAG T	2 2 2 3
3	ACTANCA 3 A C T	1 1 2 2 3 3 4 1 6 2 7 1
4	ATATA 1 ATA	1 1 3 1

Таблица 2 - Результаты тестирования для задания 2

Тест	Входные данные	Выходные данные
1	ACTANCA A\$\$A\$ \$	1
2	AAAA A\$C\$ \$	(пусто)
3	TAGTANT T&T&N& &	(пусто)
4	GATACA \$A\$A\$ \$	1

Таблица 3 - Результаты тестирования для индивидуального задания

Тест	Входные данные	Выходные данные
1	AAAA 2 AA CC	Макс цепочка suffix_link: 2 (вершина 2) Макс цепочка final_link: 1 (вершина 1)

2	NTAG 3 TAGT TAG T	Макс цепочка suffix_link: 2 (вершина 4) Макс цепочка final_link: 2 (вершина 4)
3	ACTANCA 3 A C T	Макс цепочка suffix_link: 1 (вершина 1) Макс цепочка final_link: 1 (вершина 1)
4	ATATA 1 ATA	Макс цепочка suffix_link: 2 (вершина 3) Макс цепочка final_link: 1 (вершина 1)

Приложение

main.cpp:

```
#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <algorithm>
#include <iomanip>

using namespace std;

int char_to_idx(char c) {
    switch(c) {
        case 'A': return 0;
        case 'C': return 1;
        case 'G': return 2;
        case 'T': return 3;
        case 'N': return 4;
    }
    return -1;
}

char idx_to_char(int i) {
    switch(i) {
        case 0: return 'A';
        case 1: return 'C';
        case 2: return 'G';
        case 3: return 'T';
        case 4: return 'N';
    }
    return '?';
}

struct node {
    int next[5];
    int final_link;
    int suffix_link;
```

```

    int parent;
    int pchar;
    int pattern_id;

    node() {
        for (int i = 0; i < 5; i++) {
            next[i] = -1;
        }
        suffix_link = -1;
        parent = -1;
        pattern_id = -1;
    }
};

vector<node> bor;
vector<string> patterns;
vector<int> pattern_lengths;

void print_queue(queue<int> q) {
    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
    cout << endl;
}

void print_automaton() {
    cout << "\nАВТОМАТ" << endl;
    cout << left << setw(4) << "v"
        << setw(17) << "строка"
        << setw(6) << "suff"
        << setw(8) << "final"
        << setw(18) << "терминал"
        << "переходы" << endl;
    cout << string(50, '-') << endl;

    for (int v = 0; v < bor.size(); v++) {
        cout << setw(4) << v;
    }
}

```

```

        string str = "";
        int temp = v;
        while (temp != 0) {
            str = idx_to_char(bor[temp].pchar) + str;;
            temp = bor[temp].parent;
        }
        cout << setw(12) << (str.empty() ? "root" : str);

        cout << setw(6) << bor[v].suffix_link;
        cout << setw(8) << bor[v].final_link;
        cout << setw(12) << (bor[v].pattern_id != -1 ?
patterns[bor[v].pattern_id] : "-");

        for (int c = 0; c < 5; c++) {
            if (bor[v].next[c] != -1 && bor[v].next[c] != 0) {
                cout << idx_to_char(c) << "→" << bor[v].next[c] <<
" ";
            }
        }
        cout << endl;
    }
}

void print_tree(int v = 0, string indent = "") {
    if (v == 0) {
        cout << "0 (root, suff=" << bor[0].suffix_link << ", final="
<< bor[0].final_link << ")" << endl;
    }

    vector<int> children;
    for (int c = 0; c < 5; c++) {
        if (bor[v].next[c] != -1 && bor[v].next[c] != 0) {
            children.push_back(bor[v].next[c]);
        }
    }

    for (int i = 0; i < children.size(); i++) {
        bool last = (i == children.size() - 1);
        int u = children[i];

```

```

        cout << indent;
        if (last) cout << "└─ ";
        else cout << "├─ ";

        cout << u << " (suff=" << bor[u].suffix_link
            << ", final=" << bor[u].final_link;

        if (bor[u].pattern_id != -1) {
            cout << ", term=\"" << patterns[bor[u].pattern_id] <<
"\\"";
        }
        cout << ")" << endl;

        if (last) print_tree(u, indent + "    ");
        else print_tree(u, indent + "├    ");
    }
}

```

```

void add_string(string& s, int pattern_id) {
    cout << "Добавляем в бор считанную строчку " << s << endl;

    int v = 0;
    for (char ch : s) {
        int c = char_to_idx(ch);
        if (bor[v].next[c] == -1) {
            bor[v].next[c] = bor.size();
            bor.emplace_back();
            bor.back().parent = v;
            bor.back().pchar = c;
        }

        v = bor[v].next[c];
    }
    bor[v].pattern_id = pattern_id;

    cout << "\nДобавлена \"" << s << "\"" << endl;
    print_tree();
}

```

```

}

void build_bor_suffix() {
    cout << "\n\nСоздаем очередь для BFS, чтобы построить
суффиксные и конечные ссылки в боре" << endl;
    queue<int> q;
    bor[0].final_link = 0;

    cout << "    Обрабатываем корень (первые узлы бора - алфавита
{A,C,G,T,N})" << endl;
    for (int c = 0; c < 5; c++) {
        if (bor[0].next[c] != -1) {
            int v = bor[0].next[c];
            bor[v].suffix_link = 0;
            bor[v].final_link = 0;
            q.push(v);
        } else {
            bor[0].next[c] = 0;
        }
    }

    cout << "    Присвоили первым узлам суффиксные и конечные ссылки
на корень" << endl;
    print_tree();

    while (!q.empty()) {
        int v = q.front();
        cout << "        Берем первый элемент из очереди и удаляем
его из очереди: v = " << v << endl;
        cout << "        Обрабатываем для этой ноды бора все возможные
дороги {A=0,C=1,G=2,T=3,N=4}" << endl;
        q.pop();

        for (int c = 0; c < 5; c++) {
            int child = bor[v].next[c];

```

```

        cout << "                                Вычислили ребенка по элементу с
индексом = " << c << endl;
        cout << "                                child = bor[v].next[c] = " << child
<< endl;

        if (child != -1) {
            q.push(child);
            cout << "                                Ребенок есть, добавляем
его в очередь, очередь = ";
            print_queue(q);

            cout << "                                Идем по суффиксным ссылкам
родителя (v): link = bor[v].suffix_link = " << bor[v].suffix_link <<
endl;

            int link = bor[v].suffix_link;
            while (bor[link].next[c] == -1) {
                link = bor[link].suffix_link;
                cout << "                                Так как из этой ноды
нету дороги по символу с индексом " << c << " то идем в след префиксную
ссылку: link = bor[link].suffix_link = " << bor[link].suffix_link <<
endl;
            }
            bor[child].suffix_link = bor[link].next[c];

            cout << "                                Теперь находим конечную
ссылку (v): link = bor[v].suffix_link = " << bor[v].suffix_link << endl;
            link = bor[child].suffix_link;
            if (bor[link].pattern_id != -1) {
                cout << "                                Так как нода " <<
bor[link].pattern_id << " является терминальной, то записываем ее как
конечную" << endl;
                bor[child].final_link = link;
            } else {
                cout << "                                Так как нода " <<
bor[link].pattern_id << " не является терминальной, то записываем в
конечную ссылку конечную ссылку этой ноды" << endl;
                bor[child].final_link = bor[link].final_link;
            }
}

```



```

        }
    }
}

cout << "\n\n ПОЛУЧЕННОЕ ДЕРЕВО:" << endl;
print_tree();
}

void get_max_lens() {
    cout << "\n\nВычисляем длину самой длинной цепочки из
суффиксных ссылок и самой длинной цепочки из конечных ссылок в автомате"
<< endl;

    int max_suffix_link = 0;
    int max_suffix_node = 0;

    for (int v = 1; v < bor.size(); v++) {
        cout << "\nСчитаем длину суффиксной ссылки для v = " << v
<< endl;

        int len = 0;
        int temp = v;
        while (temp != 0) {
            len++;
            temp = bor[temp].suffix_link;
        }

        cout << "    Макс длина = " << max_suffix_link << endl;
        cout << "    Найденная длина = " << max_suffix_link << endl;
        if (len > max_suffix_link) {
            cout << "        Обновляем максимально длинную суффиксную
ссылку" << endl;

            max_suffix_link = len;
            max_suffix_node = v;
        }
    }

    int max_final_link = 0;
    int max_final_node = 0;

    for (int v = 1; v < bor.size(); v++) {
        cout << "\nСчитаем длину конечной ссылки для v = " << v <<
endl;

```

```

        int len = 0;
        int temp = v;
        while (temp != 0) {
            len++;
            temp = bor[temp].final_link;
        }
        cout << "    Макс длина = " << max_suffix_link << endl;
        cout << "    Найденная длина = " << max_suffix_link << endl;
        if (len > max_final_link) {
            cout << "                Обновляем максимально длинную конечную
ссылку" << endl;
            max_final_link = len;
            max_final_node = v;
        }
    }

    cout << "\nМакс цепочка suffix_link: " << max_suffix_link << "
(вершина " << max_suffix_node << ")" << endl;
    cout << "Макс цепочка final_link: " << max_final_link << "
(вершина " << max_final_node << ")" << endl;
}

int go(int v, int c) {
    cout << "go из вершины " << v << " по символу " << c << endl;
    if (bor[v].next[c] != -1) {
        cout << "путь по такому символу из такой вершины есть,
поэтому переходим туда" << endl;
        return bor[v].next[c];
    }

    if (v == 0) {
        cout << "Вершина - нулевая и дороги нету по этому символу,
поэтому возвращаем 0" << endl;
        return 0;
    }
}

```

```

        cout << "Вершина не нулевая и дороги нету, поэтому рекурсивно
вызываем снова go на суффикс нынешней вершины bor[v].suffix_link = " <<
bor[v].suffix_link << endl;
        return go(bor[v].suffix_link, c);
    }

```

```

struct target {
    int pos;
    int pat;

    bool operator<(const target& other) const {
        if (pos != other.pos) return pos < other.pos;
        return pat < other.pat;
    }
};

```

```

int main() {
    string text;
    getline(cin, text);

    int n;
    cin >> n;
    cin.ignore();

    patterns.resize(n);
    pattern_lengths.resize(n);

    bor.emplace_back();

    for (int i = 0; i < n; i++) {
        getline(cin, patterns[i]);
        pattern_lengths[i] = patterns[i].length();
        add_string(patterns[i], i);
    }

    build_bor_suffix();
}

```

```

get_max_lens();

print_automaton();

vector<target> results;
int v = 0;

cout << "\n\nАнализируем исходный текст = " << text << endl;
for (int i = 0; i < text.length(); i++) {
    cout << "    Считываем чар = " << text[i] << endl;
    int c = char_to_idx(text[i]);
    cout << "        Переходим из вершины = " << v << " по ветке
с индексом символа = " << c << endl;
    v = go(v, c);
    cout << "        Перешли в вершину = " << v << endl;
    int temp = v;
    while (temp != 0) {
        if (bor[temp].pattern_id != -1) {
            int start_pos = i -
pattern_lengths[bor[temp].pattern_id] + 1;
            cout << "        Нашли слово = " <<
patterns[bor[temp].pattern_id] << ". Индекс вхождения = " << start_pos
<< endl;
            results.push_back({start_pos + 1,
bor[temp].pattern_id + 1});
        }
        temp = bor[temp].final_link;
    }
}

cout << "\n\nСортируем результат и выводим" << endl;
sort(results.begin(), results.end());

for (const auto& occ : results) {
    cout << occ.pos << " " << occ.pat << "\n";
}

return 0;

```

```
}
```

step-1.cpp:

```
#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <algorithm>

using namespace std;

int char_to_idx(char c) {
    switch(c) {
        case 'A': return 0;
        case 'C': return 1;
        case 'G': return 2;
        case 'T': return 3;
        case 'N': return 4;
    }
    return -1;
}

char idx_to_char(int i) {
    switch(i) {
        case 0: return 'A';
        case 1: return 'C';
        case 2: return 'G';
        case 3: return 'T';
        case 4: return 'N';
    }
    return '?';
}

struct node {
    int next[5];
    int final_link;
    int suffix_link;
    int parent;
    int pchar;
    int pattern_id;

    node() {
        for (int i = 0; i < 5; i++) {
            next[i] = -1;
        }
        suffix_link = -1;
        final_link = -1;
        parent = -1;
        pchar = -1;
        pattern_id = -1;
    }
};
```

```

vector<node> bor;
vector<string> patterns;
vector<int> pattern_lengths;

#ifdef LOG
void print_queue(queue<int> q) {
    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
    cout << endl;
}

void print_tree(int v = 0, string indent = "") {
    if (v == 0) {
        cout << "0 (root, suff=" << bor[0].suffix_link << ", final="
<< bor[0].final_link << ")" << endl;
    }

    vector<int> children;
    for (int c = 0; c < 5; c++) {
        if (bor[v].next[c] != -1 && bor[v].next[c] != 0) {
            children.push_back(bor[v].next[c]);
        }
    }

    for (int i = 0; i < children.size(); i++) {
        bool last = (i == children.size() - 1);
        int u = children[i];

        cout << indent;
        if (last) cout << "└─ ";
        else cout << "├─ ";

        cout << u << " (suff=" << bor[u].suffix_link
            << ", final=" << bor[u].final_link;

        if (bor[u].pattern_id != -1) {
            cout << ", term=\"" << patterns[bor[u].pattern_id] <<
"\\"";
        }
        cout << ")" << endl;

        if (last) print_tree(u, indent + "    ");
        else print_tree(u, indent + "    |    ");
    }
}
#endif

void add_string(string& s, int pattern_id) {
#ifdef LOG
    cout << "Добавляем в бор строку \"" << s << "\"" << endl;
#endif

    int v = 0;
    for (char ch : s) {
        int c = char_to_idx(ch);

```

```

        if (bor[v].next[c] == -1) {
            bor[v].next[c] = bor.size();
            bor.emplace_back();
            bor.back().parent = v;
            bor.back().pchar = c;
#ifdef LOG
            cout << "    Новая вершина " << bor.size() - 1 << " по
СИМВОЛУ '" << ch << "'" << endl;
#endif
        }
        v = bor[v].next[c];
    }
    bor[v].pattern_id = pattern_id;

#ifdef LOG
    cout << "    Вершина " << v << " терминал \"" << s << "\"" << endl;
    cout << "Бор после добавления:" << endl;
    print_tree();
    cout << endl;
#endif
}

void build_bor() {
#ifdef LOG
    cout << "\nПОСТРОЕНИЕ БОРА" << endl;
    cout << "Инициализация детей корня:" << endl;
#endif

    queue<int> q;
    bor[0].final_link = 0;
    bor[0].suffix_link = 0;

    for (int c = 0; c < 5; c++) {
        if (bor[0].next[c] != -1) {
            int v = bor[0].next[c];
            bor[v].suffix_link = 0;
            bor[v].final_link = 0;
            q.push(v);
#ifdef LOG
            cout << "    Вершина " << v << ": suff=0, final=0, добавлена
в очередь" << endl;
#endif
        } else {
            bor[0].next[c] = 0;
        }
    }

#ifdef LOG
    cout << "\nBFS обход:" << endl;
#endif

    while (!q.empty()) {
        int v = q.front();
        q.pop();

#ifdef LOG
        cout << "    Обрабатываем вершину " << v << " (suff=" <<
bor[v].suffix_link << ")" << endl;
#endif
    }
}

```

```

#endif

    for (int c = 0; c < 5; c++) {
        int child = bor[v].next[c];
        if (child != -1) {
            q.push(child);

            int link = bor[v].suffix_link;
            while (bor[link].next[c] == -1) {
                link = bor[link].suffix_link;
            }
            bor[child].suffix_link = bor[link].next[c];

#ifdef LOG
            cout << "    Ребёнок " << child << " по '" <<
idx_to_char(c) << "': suff=" << bor[child].suffix_link;
#endif

            link = bor[child].suffix_link;
            if (bor[link].pattern_id != -1) {
                bor[child].final_link = link;

#ifdef LOG
                cout << ", final=" << link << " (терминал \"" <<
patterns[bor[link].pattern_id] << "\"");
#endif
            } else if (link == 0) {
                bor[child].final_link = 0;

#ifdef LOG
                cout << ", final=0";
#endif
            } else {
                bor[child].final_link = bor[link].final_link;

#ifdef LOG
                cout << ", final=" << bor[child].final_link << "
(от " << link << ")";
#endif
            }

#ifdef LOG
            cout << endl;
#endif
        }
    }

}

#ifdef LOG
    cout << "\nБор построен:" << endl;
    print_tree();
#endif
}

int go(int v, int c) {
#ifdef LOG
    cout << "  go(" << v << ", '" << idx_to_char(c) << "') → ";
#endif

    if (bor[v].next[c] != -1) {
#ifdef LOG
        cout << bor[v].next[c] << " (прямой переход)" << endl;

```



```

#endif
    return bor[v].next[c];
}

    if (v == 0) {
#ifdef LOG
        cout << "0 (корень)" << endl;
#endif
        return 0;
    }

#ifdef LOG
    cout << "рекурсия через suff=" << bor[v].suffix_link << endl;
#endif
    return go(bor[v].suffix_link, c);
}

void get_max_lens() {
    int max_suffix_link = 0;
    int max_suffix_node = 0;

    for (int v = 1; v < bor.size(); v++) {
        int len = 0;
        int temp = v;
        while (temp != 0) {
            len++;
            temp = bor[temp].suffix_link;
        }
        if (len > max_suffix_link) {
            max_suffix_link = len;
            max_suffix_node = v;
        }
    }

    int max_final_link = 0;
    int max_final_node = 0;

    for (int v = 1; v < bor.size(); v++) {
        int len = 0;
        int temp = v;
        while (temp != 0) {
            len++;
            temp = bor[temp].final_link;
        }
        if (len > max_final_link) {
            max_final_link = len;
            max_final_node = v;
        }
    }

#ifdef LOG
    cout << "Макс цепочка suffix_link: " << max_suffix_link << "
(вершина " << max_suffix_node << ")" << endl;
    cout << "Макс цепочка final_link: " << max_final_link << "
(вершина " << max_final_node << ")" << endl;
#endif
}

```

```

struct target {
    int pos;
    int pat;

    bool operator<(const target& other) const {
        if (pos != other.pos) return pos < other.pos;
        return pat < other.pat;
    }
};

int main() {
    string text;
    getline(cin, text);

    int n;
    cin >> n;
    cin.ignore();

    patterns.resize(n);
    pattern_lengths.resize(n);

    bor.emplace_back();

    for (int i = 0; i < n; i++) {
        getline(cin, patterns[i]);
        pattern_lengths[i] = patterns[i].length();
        add_string(patterns[i], i);
    }

    build_bor();
    get_max_lens();

#ifdef LOG
    cout << "\nПОИСК" << endl;
    cout << "Текст: \"" << text << "\"" << endl;
#endif

    vector<target> results;
    int v = 0;

    for (int i = 0; i < text.length(); i++) {
        int c = char_to_idx(text[i]);

#ifdef LOG
        cout << "i=" << i << " '" << text[i] << "': ";
#endif

        v = go(v, c);

#ifdef LOG
        cout << "    состояние: " << v;
#endif

        int temp = v;
        while (temp != 0) {
            if (bor[temp].pattern_id != -1) {
                int start_pos = i -
pattern_lengths[bor[temp].pattern_id] + 1;

```

```

#ifdef LOG
        cout << "    НАЙДЕНО: \"" <<
patterns[bor[temp].pattern_id] << "\" (поз." << start_pos + 1 << ")";
#endif
        results.push_back({start_pos + 1, bor[temp].pattern_id
+ 1});
    }
    temp = bor[temp].final_link;
}
#ifdef LOG
    cout << endl;
#endif
}

#ifdef LOG
    cout << "\nСортируем результат" << endl;
#endif
    sort(results.begin(), results.end());

    for (auto& res : results) {
        cout << res.pos << " " << res.pat << "\n";
    }

    return 0;
}

```

step-2.cpp:

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>
#include <algorithm>

using namespace std;

int char_to_idx(char c) {
    switch(c) {
        case 'A': return 0;
        case 'C': return 1;
        case 'G': return 2;
        case 'T': return 3;
        case 'N': return 4;
    }
    return -1;
}

char idx_to_char(int i) {
    switch(i) {
        case 0: return 'A';
        case 1: return 'C';
        case 2: return 'G';
    }
}

```

```

        case 3: return 'T';
        case 4: return 'N';
    }
    return '?';
}

struct node {
    int next[5];
    int suffix_link;
    int final_link;
    int parent;
    int pchar;
    vector<int> pattern_ids;

    node() {
        for (int i = 0; i < 5; i++) {
            next[i] = -1;
        }
        suffix_link = -1;
        final_link = -1;
        parent = -1;
        pchar = -1;
    }
};

vector<node> bor;
vector<string> fragments;
vector<int> fragment_pos;

#ifdef LOG
void print_queue(queue<int> q) {
    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
    cout << endl;
}

void print_tree(int v = 0, string indent = "") {
    if (v == 0) {
        cout << "0 (root, suff=" << bor[0].suffix_link << ", final="
<< bor[0].final_link << ")" << endl;
    }

    vector<int> children;
    for (int c = 0; c < 5; c++) {
        if (bor[v].next[c] != -1 && bor[v].next[c] != 0) {
            children.push_back(bor[v].next[c]);
        }
    }

    for (int i = 0; i < children.size(); i++) {
        bool last = (i == children.size() - 1);
        int u = children[i];

        cout << indent;
        if (last) cout << "└─ ";
        else cout << "├─ ";
    }
}

```

```

        cout << u << " (suff=" << bor[u].suffix_link << ", final=" <<
bor[u].final_link;

        if (!bor[u].pattern_ids.empty()) {
            cout << ", terms=[";
            for (int j = 0; j < bor[u].pattern_ids.size(); j++) {
                if (j > 0) cout << ",";
                cout << bor[u].pattern_ids[j];
            }
            cout << "];"
        }
        cout << ")" << endl;

        if (last) print_tree(u, indent + "    ");
        else print_tree(u, indent + "|    ");
    }
}
#endif

void add_string(const string& s, int fragment_id) {
#ifdef LOG
    cout << "Добавляем фрагмент \"" << s << "\" (id=" << fragment_id
<< ")" << endl;
#endif

    int v = 0;
    for (char ch : s) {
        int c = char_to_idx(ch);
        if (bor[v].next[c] == -1) {
            bor[v].next[c] = bor.size();
            bor.emplace_back();
            bor.back().parent = v;
            bor.back().pchar = c;
#ifdef LOG
            cout << "    Новая вершина " << bor.size() - 1 << " по '" <<
ch << "'" << endl;
#endif
        }
        v = bor[v].next[c];
    }
    bor[v].pattern_ids.push_back(fragment_id);

#ifdef LOG
    cout << "    Вершина " << v << " терминал, фрагменты: [";
    for (int j = 0; j < bor[v].pattern_ids.size(); j++) {
        if (j > 0) cout << ",";
        cout << bor[v].pattern_ids[j];
    }
    cout << "]" << endl;
    cout << "Бор:" << endl;
    print_tree();
    cout << endl;
#endif
}

void build_bor() {
#ifdef LOG

```

```

    cout << "\nПОСТРОЕНИЕ БОРА" << endl;
    cout << "Инициализация детей корня:" << endl;
#endif

    queue<int> q;
    bor[0].suffix_link = 0;
    bor[0].final_link = 0;

    for (int c = 0; c < 5; c++) {
        if (bor[0].next[c] != -1) {
            int v = bor[0].next[c];
            bor[v].suffix_link = 0;
            bor[v].final_link = 0;
            q.push(v);
#ifdef LOG
            cout << "  Вершина " << v << ": suff=0, final=0, в
очередь" << endl;
#endif
        } else {
            bor[0].next[c] = 0;
        }
    }

#ifdef LOG
    cout << "\nBFS обход:" << endl;
#endif

    while (!q.empty()) {
        int v = q.front();
        q.pop();

#ifdef LOG
        cout << "  Вершина " << v << " (suff=" << bor[v].suffix_link
<< ")" << endl;
#endif

        for (int c = 0; c < 5; c++) {
            int child = bor[v].next[c];
            if (child != -1) {
                q.push(child);

                int link = bor[v].suffix_link;
                while (bor[link].next[c] == -1) {
                    link = bor[link].suffix_link;
                }
                bor[child].suffix_link = bor[link].next[c];
            }
        }

#ifdef LOG
        cout << "    child=" << child << " по '" <<
idx_to_char(c)
        << "': suff=" << bor[child].suffix_link;
#endif

        int slink = bor[child].suffix_link;
        if (!bor[slink].pattern_ids.empty()) {
            bor[child].final_link = slink;
#ifdef LOG
            cout << ", final=" << slink << " (терминал)";

```

```

#endif
        } else if (slink == 0) {
            bor[child].final_link = 0;
#ifdef LOG
            cout << ", final=0";
#endif
        } else {
            bor[child].final_link = bor[slink].final_link;
#ifdef LOG
            cout << ", final=" << bor[child].final_link << "
(от " << slink << ")";
#endif
        }
#ifdef LOG
        cout << endl;
#endif
    }
}

#ifdef LOG
    cout << "\n Бор построен:" << endl;
    print_tree();
#endif
}

int go(int v, int c) {
    if (bor[v].next[c] != -1) {
        return bor[v].next[c];
    }
    if (v == 0) {
        return 0;
    }
    return go(bor[v].suffix_link, c);
}

int main() {
    string text, pattern;
    char joker;

    getline(cin, text);
    getline(cin, pattern);
    cin >> joker;

#ifdef LOG
    cout << "Текст: \"" << text << "\"" << endl;
    cout << "Шаблон: \"" << pattern << "\"" << endl;
    cout << "Джокер: '" << joker << "'" << endl;
    cout << "Разбиваем шаблон на фрагменты" << endl;
#endif

    string current = "";

    for (int i = 0; i < pattern.length(); i++) {
        if (pattern[i] == joker) {
            if (!current.empty()) {
                fragments.push_back(current);
                fragment_pos.push_back(i - current.length());
            }
        }
    }
}

```

```

#ifdef LOG
        cout << "Фрагмент \"" << current << "\" на позиции "
<< fragment_pos.back() << endl;
#endif

        current = "";
    }
    } else {
        current += pattern[i];
    }
}

if (!current.empty()) {
    fragments.push_back(current);
    fragment_pos.push_back(pattern.length() - current.length());
#ifdef LOG
    cout << "Фрагмент \"" << current << "\" на позиции " <<
fragment_pos.back() << endl;
#endif
}

#ifdef LOG
    cout << "\nВсего фрагментов: " << fragments.size() << endl;
#endif

    bor.emplace_back();

    for (int i = 0; i < fragments.size(); i++) {
        add_string(fragments[i], i);
    }

    build_bor();

#ifdef LOG
    cout << "\nПОИСК" << endl;
#endif

    vector<int> matches(text.length() + 1, 0);
    int v = 0;

    for (int i = 0; i < text.length(); i++) {
        int c = char_to_idx(text[i]);

#ifdef LOG
        cout << "i=" << i << " '" << text[i] << "': ";
#endif

        v = go(v, c);

#ifdef LOG
        cout << "v=" << v << endl;
#endif

        int temp = v;
        while (temp != 0) {
            for (int frag_id : bor[temp].pattern_ids) {
                int frag_len = fragments[frag_id].length();
                int pattern_start = i - frag_len -
fragment_pos[frag_id] + 1;

```



```

#ifdef LOG
        cout << "    фрагмент " << frag_id << " (" <<
fragments[frag_id] << ")", pattern_start=" << pattern_start;
#endif

        if (pattern_start >= 0 && pattern_start +
pattern.length() <= text.length()) {
            matches[pattern_start]++;
#ifdef LOG
                cout << " OK, matches[" << pattern_start << "]="
<< matches[pattern_start];
#endif

                if (matches[pattern_start] ==
(int) fragments.size()) {
#ifdef LOG
                        cout << "\nНашлось совпадение всей строчки!"
<< endl;
#endif
                        cout << pattern_start + 1 << "\n";
                }
        }
#ifdef LOG
        cout << endl;
#endif
    }
    temp = bor[temp].final_link;
}

return 0;
}

```